

Rentsure - online direct property dealing application.

MAJOR PROJECT REPORT

**Submitted in partial fulfillment of the requirement for the Degree of
Bachelors of Engineering in Computer Science & Engineering**

Submitted To:



PARUL UNIVERSITY, VADODARA, GUJARAT (INDIA)

Submitted By:

**Sahil Roy(2303051050795)
Subodh Tharu(2303051050923)
Aditya patel(2303051051442)**

Under The Guidance of:

**Kinjal Kevin Gandhi
(Assistant Professor, Computer Science & Engineering)**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PARUL INSTITUTE OF TECHNOLOGY VADODARA, GUJARAT

SESSION: AY 2026-2027

Parul Institute of Technology



(Session: 2025 -2026)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that _____ Students of **CSE VII** Semester of “ Parul Institute of Technology, Vadodara” has completed their Major Project titled “Rentsure - online direct property dealing application.”, as per the syllabus and has submitted a satisfactory report on this project as a partial fulfillment towards the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** under **Parul University, Vadodara, Gujarat (India)**.

ARNIKA PATEL

(Project Coordinator)

External Examiner

Kinjal Kevin Gandhi
(Project Guide)
(Assistant Professor)
(CSE / IT)

Prof. Sumitra Menaria
Head (CSE)
PIT, Vadodara

DECLARATION

I, the undersigned, solemnly declare that the project report titled "RentSure - Online property dealing web application." is based on my own work carried out during the course of my study under the supervision of Kinjal Kevin Gandhi, Assistant Professor, Computer Science & Engineering .

I assert that the statements made and conclusions drawn are the outcomes of my own work. I further certify that:

1. The work contained in the report is original and has been done by us under the general supervision of our supervisor.
2. The work has not been submitted to any other Institution for any other degree / diploma/ certificate in this university or any other University of India or abroad.
3. I have followed the guidelines provided by the university in writing the report.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us throughout the development of the project “EstateHub – A Real Estate Property Listing and Rental Platform”. This project would not have been possible without the encouragement, guidance, and cooperation extended to us.

We are extremely thankful to our project guide, Asst. Prof. Kinjal Kevin Gandhi, Computer Science & Engineering Department, for her constant support, valuable suggestions, and patient guidance during every stage of this project. Her insights helped us overcome numerous technical challenges and improve the overall quality of our work.

We would also like to extend our gratitude to Prof. Sumitra Menaria, Head of the Computer Science & Engineering Department, and to Parul Institute of Technology, Parul University, Vadodara, for providing us with the resources and opportunity to carry out this project.

Finally, we thank our family and friends for their continuous encouragement and motivation, which helped us complete this project successfully.

Place: Vadodara

Date: _____

Contents

S. No	Particulars	Page No
1	Acknowledgement	4
2	Abstract	9
3	Chapter – 1 Introduction	10
4	1.1 General Introduction	10
5	1.2 Problem Definition	10
6	1.3 Motivation	10
7	1.4 Objectives	10
8	1.5 Scope of the Project	10
9	1.6 Hardware & Software Requirement	11
10	Chapter – 2 Literature Survey	12
11	2.1 Review of Existing Property Portals	12
12	2.2 Analysis of Existing Systems	12
13	2.3 Background Technology	12
14	Chapter – 3 Methodology	13
15	3.1 Existing Methodology	13
16	3.2 Proposed Methodology	13
17	Chapter – 4 Diagrams	14
18	4.1 Data Flow Diagram (DFD)	14
19	4.2 Use Case Diagram	14
20	4.3 E-R Diagram	15
21	Chapter – 5 Implementation / GUI	16
22	5.1 Module Description	16
23	5.2 Pseudo Code of Module	16
24	5.3 Screenshot of Module	16
25	Chapter – 6 Results	19
26	Chapter – 7 Conclusion and Future Scope	20
27	7.1 Conclusion	20
28	7.2 Future Scope	20
29	References	21

LIST OF TABLES

S.No	Table No.	Table Description	Page No.
1	Table 1.1	Hardware & Software Requirements	11
2	Table 2.1	Technology Stack Overview	12
3	Table 3.1	REST API Endpoints	13
4	Table 4.1	Use Case Summary	14

LIST OF FIGURES

S.No	Figure No.	Name of Figure	Page No.
1	Figure 4.1	Data Flow Diagram – Level 0 (Context Diagram)	14
2	Figure 4.2	Data Flow Diagram – Level 1	14
3	Figure 4.3	Use Case Diagram	14
4	Figure 4.4	Entity-Relationship Diagram	15
5	Figure 5.1	Home Page	16
6	Figure 5.2	Latest Listings Section	17
7	Figure 5.3	Platform Features & Call-to-Action	17
8	Figure 5.4	Admin Dashboard	18

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Full Form
1	MERN	MongoDB, Express.js, React, Node.js
2	JWT	JSON Web Token
3	API	Application Programming Interface
4	SPA	Single Page Application
5	CRUD	Create, Read, Update, Delete
6	GUI	Graphical User Interface
7	DFD	Data Flow Diagram
8	ER	Entity-Relationship
9	REST	Representational State Transfer
10	UI/UX	User Interface / User Experience

ABSTRACT

The primary objective of this project is to design and develop EstateHub, a production-ready, full-stack real estate property listing and rental platform. Built utilizing the MERN stack (MongoDB, Express.js, React 18, Node.js), the platform addresses the inefficiencies in traditional property hunting by providing a centralized, role-based ecosystem.

The system supports three distinct user roles: Buyers (who can search, filter, save favorites, and send inquiries), Owners (who manage listings and reply to inquiries), and Admins (who oversee platform operations, approve listings, and manage users). Core functionalities include advanced search filtering, JWT-based authentication, interactive Google Maps integration for property location, and dual-view (Grid and Map) browsing.

EstateHub aims to streamline real estate transactions, offering a highly responsive, secure, and user-friendly digital environment for modern property management.

Chapter – 1

Introduction

1.1 General Introduction

The rapid digitization of services has fundamentally transformed how individuals buy, sell, and rent real estate. EstateHub is a comprehensive web application designed to bridge the gap between property owners and prospective buyers or tenants. By leveraging modern web technologies, the platform delivers a seamless, secure, and interactive property browsing experience.

1.2 Problem Definition

Traditional property hunting relies heavily on physical brokers, fragmented local listings, or platforms lacking rigorous verification. Users frequently face issues such as outdated property data, lack of geographical context (maps), poor communication channels with owners, and overwhelming, unfiltered search results.

1.3 Motivation

The motivation behind EstateHub is to create an intuitive, high-performance platform that empowers users with direct access to structured real estate data. By implementing a role-based access control system and integrating interactive mapping, the platform aims to eliminate middlemen and foster transparent communication between owners and buyers.

1.4 Objectives

- To develop a secure, role-based authentication system (Buyer, Owner, Admin) using JWT and bcryptjs.
- To implement a dynamic property listing module supporting image URLs, amenities, and floor plans.
- To create an advanced search engine with multi-parameter filtering (city, type, price, bedrooms, furnishing).
- To integrate Google Maps API for spatial visualization of properties.

1.5 Scope of the Project

The scope encompasses the full software development lifecycle of a web-based application. It serves property seekers (Buyers), property listers (Owners), and system moderators (Admins). The application is accessible via standard web browsers and is optimized for responsive, mobile-first performance across all device sizes.

1.5.1 Existing System

Existing systems often consist of either highly localized, manual broker networks or outdated web portals that lack modern UI/UX principles. Many current solutions do not offer dual grid/map views, lack built-in threading for inquiries, or fail to provide a dedicated admin dashboard for listing moderation.

1.5.2 Proposed System

EstateHub proposes a centralized, MERN-stack architecture. It introduces a mandatory admin-approval workflow for new listings (defaulting to a pending state) to ensure quality control. Furthermore, it

incorporates real-time notifications via react-hot-toast and a seamless communication thread between buyers and owners.

1.6 Hardware & Software Requirement

- Hardware: Standard PC/Mac, Minimum 8GB RAM.
- Software: Node.js (v18+), MongoDB (Local or Atlas), npm/yarn, Code Editor (VS Code), React 18, Vite, Tailwind CSS.

Category	Component	Details
Hardware	System	Standard PC/Mac, Minimum 8GB RAM
Software	Runtime	Node.js (v18+)
Software	Database	MongoDB (Local or Atlas)
Software	Package Manager	npm / yarn
Software	Code Editor	VS Code
Software	Frontend Framework	React 18
Software	Build Tool	Vite
Software	Styling	Tailwind CSS

Table 1.1: Hardware & Software Requirements

Chapter – 2

Literature Survey

2.1 Review of Existing Property Portals

Current real estate applications focus heavily on aggregating large volumes of data but often compromise on user experience. Systems studied prior to development indicated a strong need for visually appealing interfaces and precise location-based tracking.

2.2 Analysis of Existing Systems

- Advantages: Broad reach, large user bases, established market presence.
- Disadvantages: High latency, complex navigation, presence of spam/unverified listings, and lack of direct owner-to-buyer communication channels.
- Limitations: Many platforms restrict API integrations or charge premium fees for basic geographic visualization.

2.3 Background Technology

EstateHub is built on the MERN stack.

- React 18 & Vite: Provide a fast, component-based frontend architecture.
- Node.js & Express.js: Handle asynchronous backend requests and API routing.
- MongoDB & Mongoose: Provide a flexible, document-based NoSQL database schema ideal for varying property attributes.
- Google Maps API: Used via @react-google-maps/api to render property markers dynamically.

Layer	Technology	Purpose
Frontend	React 18 & Vite	Fast, component-based frontend architecture
Backend	Node.js & Express.js	Handles asynchronous backend requests and API routing
Database	MongoDB & Mongoose	Flexible, document-based NoSQL schema ideal for varying property attributes
Maps	Google Maps API (@react-google-maps/api)	Renders property markers dynamically

Table 2.1: Technology Stack Overview

Chapter – 3

Methodology

3.1 Existing Methodology

Traditionally, real estate transactions require manual data entry by agents, physical site visits just to verify basic amenities, and disparate communication via phone or external email, leading to a disconnected user journey.

3.2 Proposed Methodology

EstateHub adopts an Agile development methodology. The system architecture follows a Client-Server model connected via RESTful APIs.

Frontend: Built with React Router v6 for seamless Single Page Application (SPA) navigation. State is managed via React Context (AuthContext.jsx). Axios is utilized as the HTTP client intercepting requests to inject JWT tokens.

Backend: Express routes (/api/auth, /api/properties, /api/inquiries) handle specific domain logic through dedicated controllers (authController.js, propertyController.js). Data persistence is handled via Mongoose models.

Endpoint	Controller	Description
/api/auth	authController.js	Handles JWT-based authentication, registration, and login
/api/properties	propertyController.js	Handles property CRUD operations and filtered search queries
/api/inquiries	Mongoose models (Inquiry)	Links Buyer, Owner, and Property to facilitate message threading

Table 3.1: REST API Endpoints

Chapter – 4

Diagrams

4.1 Data Flow Diagram (DFD)

- Level 0 (Context Diagram): Shows the core EstateHub system interacting with the three external entities: Buyer, Owner, and Admin.
- Level 1: Breaks down the system into core processes: Authentication, Property Management, Inquiry Processing, and Admin Analytics.



Figure 4.1: Data Flow Diagram – Level 0 (Context Diagram)

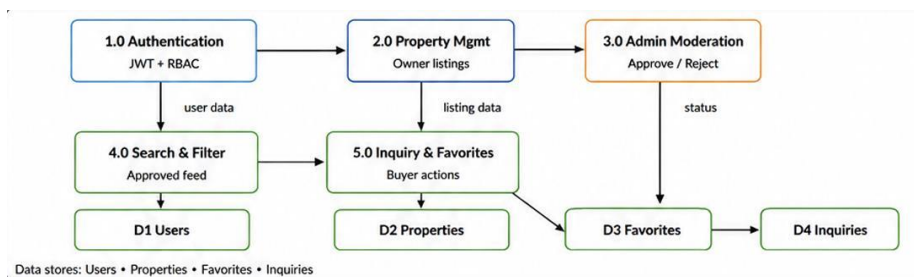


Figure 4.2: Data Flow Diagram – Level 1

4.2 Use Case Diagram

- Buyer: Register/Login, Search Properties, Apply Filters, Save Favorites, Send Inquiries.
- Owner: Register/Login, Create Listing, Edit/Delete Listing, View Inquiries, Reply to Inquiries.
- Admin: View Dashboard Stats, Manage Users, Approve/Reject Pending Properties.

Actor	Key Use Cases
Buyer	Register/Login, Search Properties, Apply Filters, Save Favorites, Send Inquiries
Owner	Register/Login, Create Listing, Edit/Delete Listing, View Inquiries, Reply to Inquiries
Admin	View Dashboard Stats, Manage Users, Approve/Reject Pending Properties

Table 4.1: Use Case Summary

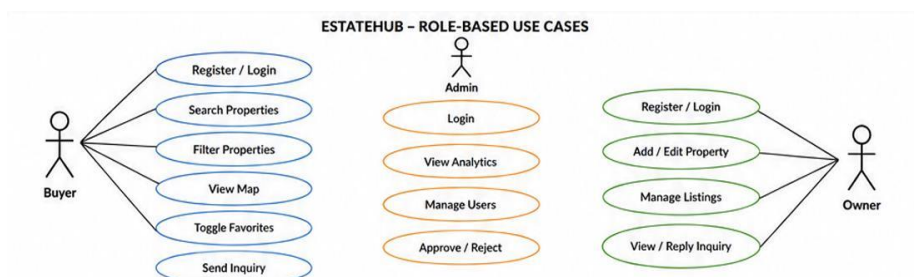


Figure 4.3: Use Case Diagram

4.3 E-R Diagram

The database consists of four primary collections: Users, Properties, Favorites, and Inquiries. The E-R diagram maps the One-to-Many relationship between Users (Owners) and Properties, and the Many-to-Many relationships handled by the Favorites junction schema.

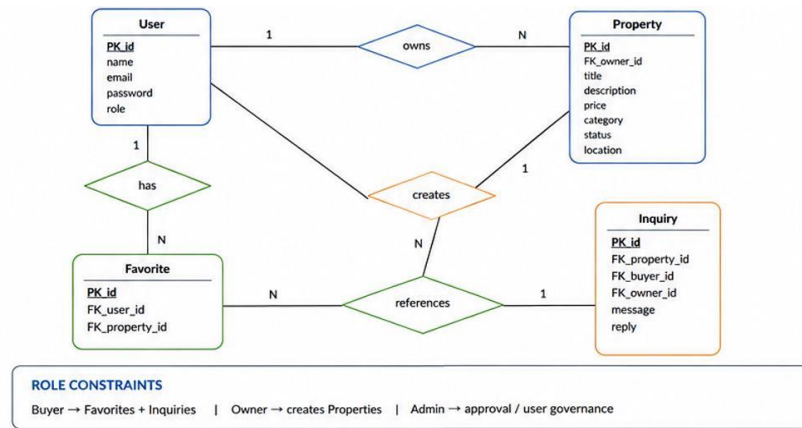


Figure 4.4: Entity-Relationship Diagram

Chapter – 5

Implementation / GUI

5.1 Module Description

- Auth Module: Handles JWT generation and password hashing (bcryptjs). Grants role-based access.
- Property Module: Allows fetching paginated properties. Includes a complex filtering algorithm querying MongoDB based on minPrice, maxPrice, bedrooms, and furnishing.
- Inquiry Module: Links a Buyer, an Owner, and a Property ID to facilitate message threading.
- Admin Module: Provides platform-wide analytics and status toggling for user-submitted properties.

5.2 Pseudo Code of Module (Search Filter Example)

```

FUNCTION fetchProperties(query_parameters)
  INITIALIZE empty filter_object
  IF query_parameters.city EXISTS THEN add to filter_object
  IF query_parameters.minPrice OR maxPrice EXISTS THEN
    add price range logic to filter_object
  END IF
  EXECUTE DB.Properties.find(filter_object).paginate(page, limit)
  RETURN results
ENDFUNCTION
  
```

5.3 Screenshot of Module

The following screens illustrate the primary interfaces of the EstateHub platform.

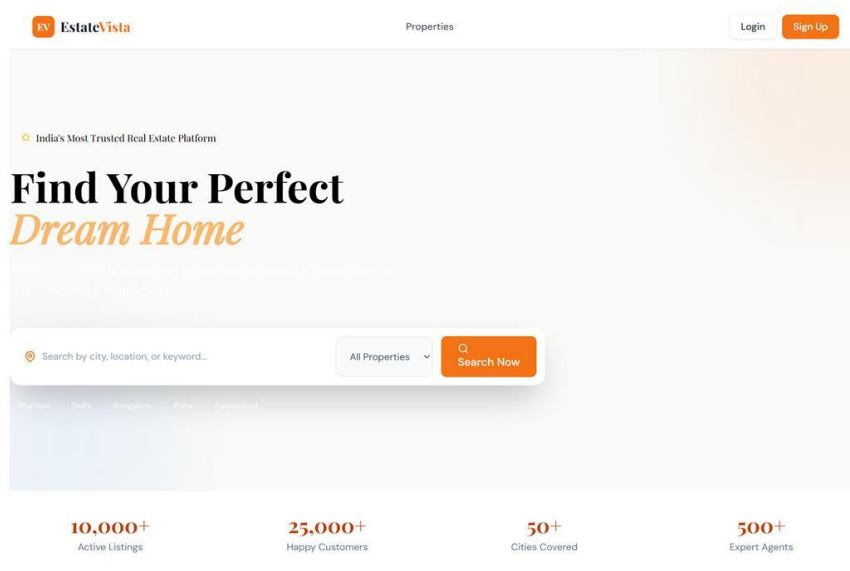


Figure 5.1: Home Page

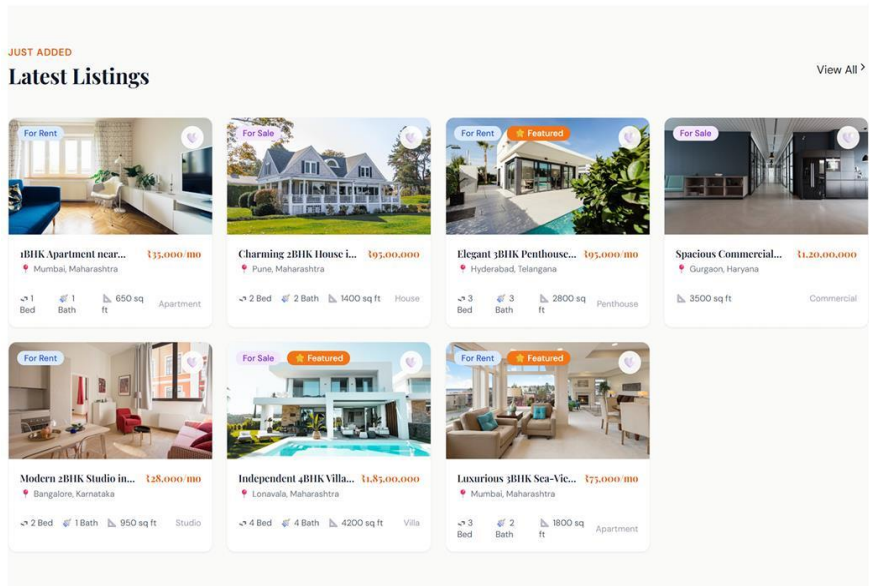


Figure 5.2: Latest Listings Section

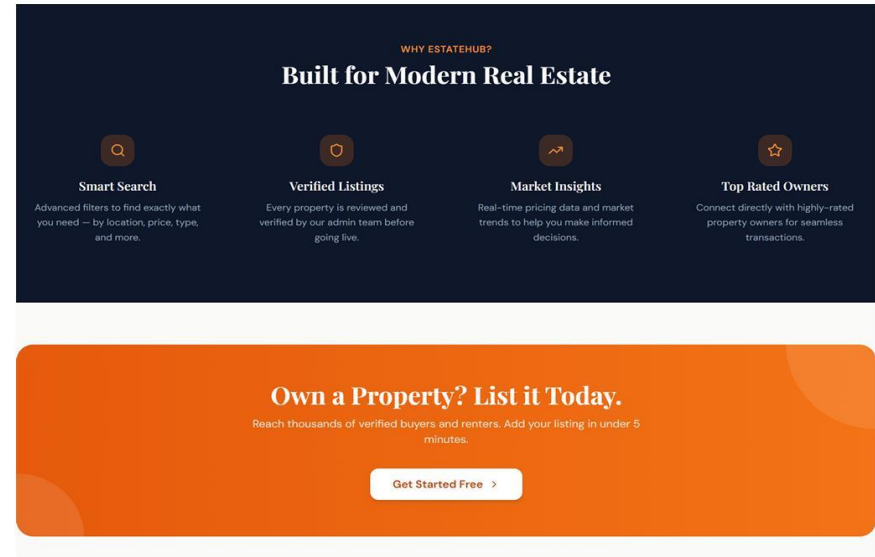


Figure 5.3: Platform Features & Call-to-Action

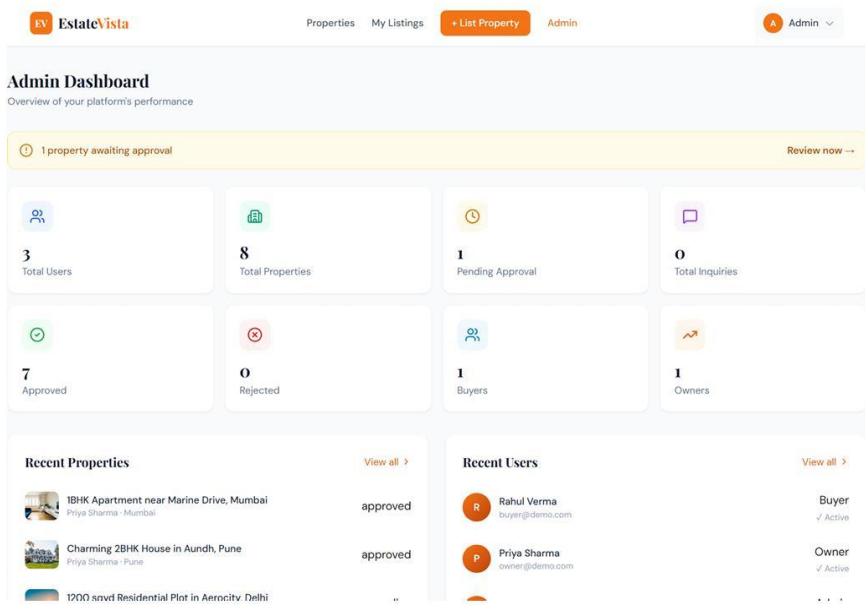


Figure 5.4: Admin Dashboard

Chapter – 6

Results

The implementation of EstateHub yielded a highly responsive, full-stack SPA. The API routes efficiently handle standard CRUD operations with average response times well within acceptable web standards. The role-based routing successfully restricts unauthorized access (e.g., Buyers cannot access /api/admin/stats).

The integration of the Google Maps API successfully plots properties based on stored coordinates, enhancing the spatial context for prospective buyers. The application is successfully prepared for cloud deployment (Frontend to Vercel, Backend to Render, Database to MongoDB Atlas).

Chapter – 7

Conclusion and Future Scope

7.1 Conclusion

EstateHub successfully demonstrates the capability of modern JavaScript frameworks to solve real-world logistical problems in the real estate sector. By enforcing an admin-approval system and integrating direct inquiries, the platform provides a secure, curated, and efficient marketplace for property transactions.

7.2 Future Scope

Future iterations of EstateHub could integrate 3D virtual property tours, a built-in payment gateway for digital lease signing and deposit handling, and real-time WebSocket chat functionality to replace the current threaded inquiry system. Additionally, machine learning algorithms could be introduced to recommend properties based on a user's viewing history.

REFERENCES

- [1] React Documentation. (2023). React: The library for web and native user interfaces. [Online] Available at: <https://react.dev/>
- [2] Node.js Documentation. (2023). Node.js v18.x Documentation. [Online] Available at: <https://nodejs.org/docs/latest-v18.x/api/>
- [3] MongoDB. (2023). MongoDB Atlas Node.js Quick Start. [Online] Available at: <https://www.mongodb.com/docs/drivers/node/current/quick-start/>
- [4] Google Maps Platform. (2023). Maps JavaScript API Documentation. [Online] Available at: <https://developers.google.com/maps/documentation/javascript>